

CORSO DI CYBERSECURITY – EPICODE

BUILD WEEK

Buffer Overflow su oscp.exe – OVERFLOW7

Build Week

A cura di:

**Akatsuki: Nicola Madonna, Lorenzo
Rizzuti, Donato Barbetta, Alex Tarantino,
Leonardo Onori**

Epicode

9 settembre 2026

Indice

1	Riassunto	2
2	Obiettivi	2
3	Configurazione del Laboratorio	3
3.1	Ambiente di laboratorio	3
3.2	Configurazione di rete	3
3.3	Verifica del servizio	3
4	Analisi dei Dati Raccolti	4
4.1	Determinazione dell'offset EIP	4
4.2	Identificazione dei badchar	4
4.3	Individuazione del gadget JMP ESP	4
4.4	Costruzione dello shellcode	5
4.5	Prova di Concetto – Esecuzione di codice remoto	5
4.6	Limitazioni osservate	6
5	Conclusioni	7

1 Riassunto

Questo report documenta lo sfruttamento di una vulnerabilità di buffer overflow presente nel binario `oscp.exe`, in ascolto sulla porta 1337 su una macchina Windows 10, tramite il comando `OVERFLOW7`. Utilizzando Immunity Debugger con il plugin Mona, è stato determinato l'offset esatto a cui l'input sovrascrive il registro EIP, identificati iterativamente i caratteri non ammessi dal buffer (badchar), individuato un indirizzo di ritorno affidabile (`jmp esp`) privo di protezioni di sicurezza attive, e costruito uno shellcode Windows compatibile tramite `msfvenom`. L'exploit finale ha prodotto una reverse shell funzionante sulla macchina vittima, confermando l'esecuzione di codice remoto (RCE).

2 Obiettivi

- Individuare l'offset esatto a cui l'input del comando `OVERFLOW7` sovrascrive il registro EIP.
- Identificare i badchar che il buffer di `oscp.exe` non gestisce correttamente.
- Individuare un indirizzo di ritorno (`jmp esp`) privo di ASLR, Rebase, SafeSEH, CFG e altre protezioni.
- Generare uno shellcode compatibile con i vincoli individuati.
- Costruire ed eseguire un exploit completo per ottenere l'esecuzione di codice remoto sulla macchina vittima.

3 Configurazione del Laboratorio

3.1 Ambiente di laboratorio

Macchina vittima: Windows 10 Pro (build 10240), con `oscp.exe` in esecuzione all'interno di Immunity Debugger (plugin Mona per l'analisi della memoria). Macchina attaccante: Kali Linux, con `netcat`, `pattern_create.rb`/`pattern_offset.rb` (Metasploit Framework) e `msfvenom` per la generazione dello shellcode.

3.2 Configurazione di rete

Le due macchine comunicano su una rete interna VirtualBox. Target Windows: 192.168.200.101:1337. Attaccante Kali: 192.168.200.100.

3.3 Verifica del servizio

Connettendosi al target con `netcat` si riceve un banner che conferma il servizio vulnerabile e i comandi disponibili:

```
nc 192.168.200.101 1337
Welcome to OSCP Vulnerable Server! Enter HELP for help.
HELP
Valid Commands:
HELP
OVERFLOW1 [value]
...
OVERFLOW7 [value]
...
EXIT
```

4 Analisi dei Dati Raccolti

4.1 Determinazione dell'offset EIP

Un primo test con un buffer di 3000 caratteri "A" ha prodotto un crash anomalo: *"Access violation when writing to [00D40000]"* durante una `strcpy` interna di `msvcrt.dll`, anziché il classico `EIP=41414141`. L'analisi del call stack (Immunity Debugger, Alt+K) ha rivelato che il buffer di destinazione dispone di soli ~2816 byte di spazio scrivibile prima di raggiungere una zona di memoria non mappata. Riducendo il buffer a 2000 caratteri si è ottenuto il crash "pulito" atteso, permettendo di procedere con `pattern_create`.

Tabella 1: Calcolo e verifica dell'offset EIP per OVERFLOW7.

Parametro	Valore
Lunghezza pattern (<code>pattern_create</code>)	2048 byte
Valore EIP dopo il crash	0x72423572
Conversione little-endian	r5Br
Offset calcolato (<code>pattern_offset</code>)	1306 byte
Verifica PoC – EIP (atteso 42424242)	42424242
Verifica PoC – ESP (atteso sequenza di "C")	CCCCCCCC...

4.2 Identificazione dei badchar

I badchar sono stati individuati inviando in sequenza tutti i 256 byte possibili dopo l'offset e confrontando, tramite Mona, la memoria effettiva con un riferimento pulito. Il processo è stato ripetuto escludendo un byte alla volta, per isolare i falsi positivi causati dall'effetto a cascata di un badchar reale sui byte successivi.

Tabella 2: Iterazioni di identificazione dei badchar per OVERFLOW7.

Iter.	Byte escluso	Corruzione dopo	Candidati residui
1	\x00	139 byte	8c 8d ae af be bf fb
2	\x8c	172 byte	ae be bf fc
3	\xae	187 byte	be bf fb fc
4	\xbe	247 byte	fb fc
5	\xfb	–	<i>nessuna corruzione</i>

Lista finale badchar: `\x00 \x8c \xae \xbe \xfb` (5 byte), confermata da Mona con il messaggio *"Hooray, normal shellcode unmodified"*.

4.3 Individuazione del gadget JMP ESP

Tramite `!mona jmp -r esp -cpb` sono stati individuati 9 indirizzi validi, tutti privi di protezioni attive:

Tabella 3: Gadget JMP ESP selezionato per l'exploit.

Indirizzo	Modulo	Protezioni
0x625011af (scelto)	essfunc.dll	ASLR/Rebase/SafeSEH/CFG/OS: False

4.4 Costruzione dello shellcode

Lo shellcode è stato generato con `msfvenom`. Con questi badchar l'encoder di default `x86/shikata_ga_nai` ha fallito (*"A valid opcode permutation could not be found"*); `msfvenom` è passato automaticamente all'encoder `x86/call14_dword_xor`, riuscito con successo.

Tabella 4: Parametri dello shellcode generato per OVERFLOW7.

Parametro	Valore
Payload	windows/shell_reverse_tcp
LHOST	192.168.200.100
LPORT	1234
Encoder utilizzato	x86/call14_dword_xor
Dimensione shellcode	348 byte

4.5 Prova di Concetto – Esecuzione di codice remoto

Il payload finale (padding da 1306 byte + indirizzo di ritorno `0x625011af` + NOP sled di 32 byte + shellcode) è stato inviato al target preceduto dal comando `OVERFLOW7`. La connessione ricevuta sul listener `netcat` in ascolto sulla macchina Kali conferma l'esecuzione di codice remoto:

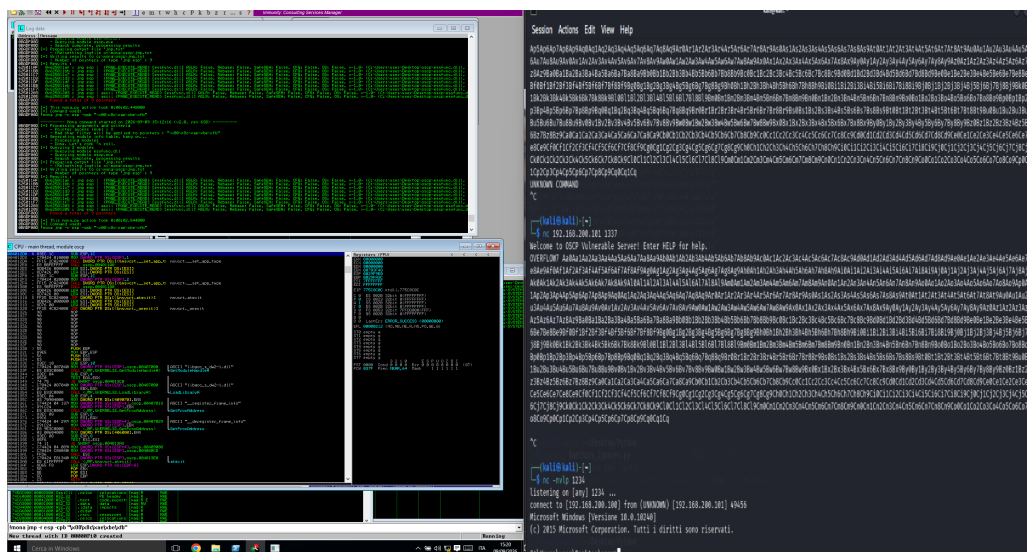


Figura 1: Reverse shell ricevuta sulla porta 1234 dopo l'invio dell'exploit OVERFLOW7.

4.6 Limitazioni osservate

A differenza degli altri comandi della stessa famiglia (es. OVERFLOW1, OVERFLOW8), il buffer gestito da OVERFLOW7 presenta uno spazio di scrittura limitato (~2816 byte) prima di una zona di memoria non mappata: payload troppo lunghi causano un crash da scrittura anomala anziché il classico overwrite di EIP, richiedendo una riduzione mirata della dimensione del buffer di fuzzing iniziale.

5 Conclusioni

Lo sfruttamento del comando **OVERFLOW7** di **oscp.exe** ha richiesto un adattamento della metodologia standard a causa del limite di scrittura del buffer di destinazione, dimostrando l'importanza di osservare attentamente il comportamento del crash prima di procedere meccanicamente con **pattern_create**. Offset, badchar e gadget sono risultati specifici di questo comando e distinti da quelli di altri **OVERFLOW** testati sullo stesso binario, confermando che ogni punto di ingresso vulnerabile va analizzato individualmente anche a parità di applicazione target.

L'exploit finale ha raggiunto con successo l'esecuzione di codice remoto, dimostrando concretamente l'impatto di una vulnerabilità di buffer overflow non mitigata: assenza di ASLR, SafeSEH e altre protezioni di memoria rende l'intera catena di exploitation deterministicamente riproducibile.